

Robustness against Consistency Models with Atomic Visibility

Giovanni Bernardi and Alexey Gotsman

IMDEA Software Institute, Madrid, Spain

Abstract

To achieve scalability, modern Internet services often rely on distributed databases with consistency models for transactions weaker than serializability. At present, application programmers often lack techniques to ensure that the weakness of these consistency models does not violate application correctness. We present criteria to check whether applications that rely on a database providing only weak consistency are *robust*, i.e., behave as if they used a database providing serializability. When this is the case, the application programmer can reap the scalability benefits of weak consistency while being able to easily check the desired correctness properties. Our results handle systematically and uniformly several recently proposed weak consistency models, as well as a mechanism for strengthening consistency in parts of an application.

1998 ACM Subject Classification C.2.4 Distributed Systems

Keywords and phrases Robustness, Replication, Consistency models, Transactions

1 Introduction

To achieve scalability and availability, modern Internet services often rely on large-scale databases that replicate and partition data across a large number of nodes and/or a wide geographical span (e.g., [3, 4, 5, 6, 10, 11, 14, 20, 22, 26, 27]). The database clients invoke transactions on the data at any of the nodes, and the nodes communicate changes to each other using message passing. Ideally, we want this distributed system to provide strong guarantees about transaction processing, such as *serializability* [8]: the results of concurrently executing a set of transactions could be obtained if these transactions were executed serially in some order. Serializability is useful because it allows an application programmer to easily establish desired correctness properties. For example, to check that the transactions of an application preserve a given data integrity constraint, the programmer only needs to check that every transaction does so when executed in isolation, without worrying about concurrency. Unfortunately, achieving serializability requires excessive synchronisation among database nodes, which slows down the database and even makes it unavailable if network connections between replicas fail [1, 17]. For this reason, nowadays large-scale databases often provide weak consistency guarantees, which allow non-serializable behaviours, called *anomalies*.

As a motivating example, consider a toy on-line auction application with transactions defined by the *transactional programs* in Figure 1. The program `RegUser` creates a new user account. It manipulates the table `USERS`, whose rows contain a primary key (`uId`) and a nickname. An invocation of `RegUser(uname)` inserts a new row in `USERS` only if the nickname `uname` does not appear in `USERS`, to ensure that nicknames are unique. The program `ViewUsers` can be used to view all the users. Some databases [3, 22, 26] may allow executions of `RegUser` and `ViewUsers` such as the one sketched in Figure 2(c). There two invocations of `RegUser` generate the transactions T_1 and T_2 ; these write two rows of `USERS`, denoted by x and y , to register the users *Alice* and *Bob*. The program `ViewUsers()` then is



© Giovanni Bernardi and Alexey Gotsman;

licensed under Creative Commons License CC-BY

27th International Conference on Concurrency Theory (CONCUR 2016).

Editors: Josée Desharnais and Radha Jagadeesan; Article No. 07; pp. 07:1–07:14

Leibniz International Proceedings in Informatics



LIPIC Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

RegUser(uname) select name from USERS as ret_name where name = uname; if ret_name defined then abort else insert into USERS values (new uId, uname);	StoreBid(iid,val) insert into BIDS values (new bId, iid, val); select nbids from ITEMS as n where iid = iid; update ITEMS set nbids = n + 1 where iid = iid;	ViewItem(iid) select * from ITEMS as ret_item where iid = iid;
		ViewUsers() select * from USERS as ret_users;
		USERS(<u>uId</u> , name) ITEMS(<u>iId</u> , desc, nbids) BIDS(<u>bId</u> , iid, val)

■ **Figure 1** SQL code and table schema for an auction application. Primary keys are underlined.

invoked twice; the invocation in T_3 sees *Alice* but not *Bob*, while the invocation in T_4 sees *Bob* but not *Alice*. This result, called a *long fork* anomaly, cannot be obtained by executing the four transactions in any sequence and, hence, is not serializable.

The past few years have seen a number of proposals of new transactional consistency models for modern large-scale databases [3, 5, 11, 22, 26], differing in how much they weaken consistency, by exposing such anomalies, in exchange for improved performance. Unfortunately, application programmers often lack techniques to ensure that the weakness of these consistency models does not violate application correctness. This situation hinders the adoption of the novel consistency models by mainstream database developers and application programmers.

One way to address this problem is using the notion of application *robustness* [9, 15, 16]. An application is robust against a particular weak consistency model if it behaves the same whether using a database providing this model or serializability. If an application is robust against a given weak consistency model, then programmers can reap the performance benefits of using weak consistency while being able to easily check the desired correctness properties.

In this paper we develop criteria for checking the robustness of applications against three recently proposed consistency models—causal (aka causal+) consistency (CC) [22], prefix consistency [11] (PC) and parallel snapshot isolation [26] (PSI, aka non-monotonic snapshot isolation [3]). As a corollary of our results, we also derive an existing robustness criterion [16] for a classical model of snapshot isolation [6] (SI). Our criteria also handle variants of the consistency models that allow application programmers to request that certain transactions be executed under serializability and thereby ensure the robustness of applications that are not robust otherwise.

We handle the above four consistency models in a uniform and systematic way by exploiting a recently proposed framework [12] for declaratively specifying their semantics (§2). In particular, all of the consistency models that we consider guarantee the *atomic visibility* of transactions: either all or none of the writes performed by a transaction can be observed by other transactions. This allows us to simplify reasoning needed to establish robustness criteria by abstracting from internals of transactions in application executions. We first propose a *dynamic* robustness criterion that checks whether a given execution is serializable (§3). We formulate this criterion in terms of the dependency graph of the execution [2], describing several kinds of relationships between its transactions: an execution is serializable if its dependency graph contains no cycles of a certain form, which we call *critical*. Criteria for robustness against different consistency models differ in which cycles are considered critical. We then illustrate how our dynamic robustness criteria on a single execution can be lifted

to *static* criteria that check that all executions of a given application are serializable (§4).

2 Consistency Model Specifications

We start by recalling from [12] a formal model of database computations and the specifications of the consistency models that we handle. These specifications are declarative, which greatly simplifies our formal development. Nonetheless, as shown in [12], the specifications are equivalent to certain operational specifications, close to implementations.

We consider a database storing **objects** $\text{Obj} = \{x, y, \dots\}$, which we assume to be natural-valued. Clients interact with the database by issuing **read** and **write** operations on the objects, grouped into transactions. We denote each operation invocation by an **event** (ι, o) , where ι is an identifier from a denumerable set EventId , and $o \in \{\text{read}(x, n), \text{write}(x, n) \mid x \in \text{Obj}, n \in \mathbb{N}\}$ describes the operation invoked and its outcome: reading a value n from an object x or writing n to x . We range over events by e, f, g and denote the set of all events by Event . In the following we denote irrelevant expressions by $_$, and write $e \vdash \text{write}(x, n)$ if $e = (_, \text{write}(x, n))$ and $e \vdash \text{read}(x, n)$ if $e = (_, \text{read}(x, n))$. A binary relation $<$ is a **strict partial order** if it is transitive and irreflexive. It is **total** if additionally for all elements a and b , we have $a < b$, $b < a$ or $a = b$.

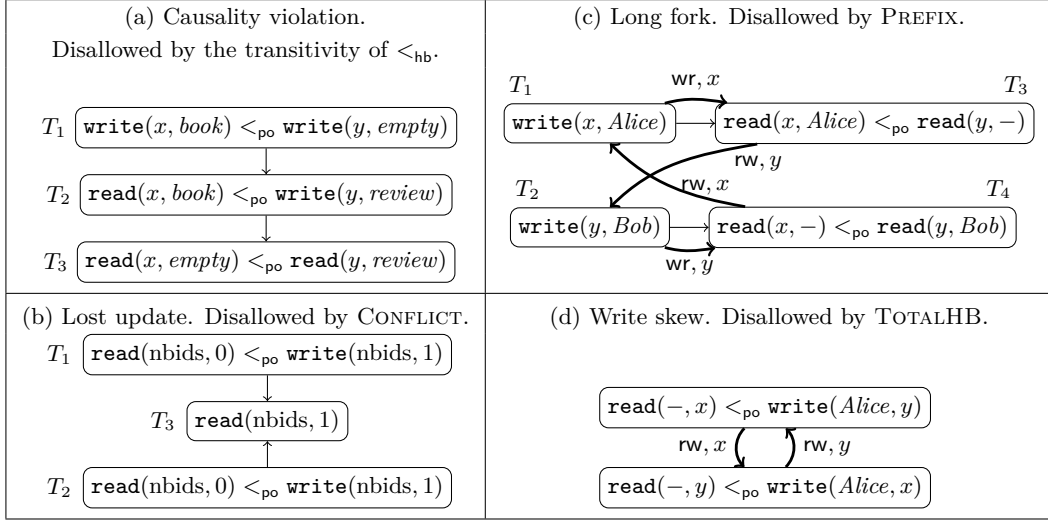
► **Definition 1.** A **transaction** $T, S, \dots$ is a pair $(E, <_{\text{po}})$, where $E \subseteq \text{Event}$ is a finite, non-empty set of events with distinct identifiers, and the **program order** $<_{\text{po}}$ is a total order over E . A **history** H is a finite set of transactions with disjoint sets of event identifiers. An annotated history (H, level) is a pair where H is an history and $\text{level} : H \rightarrow \{\text{SER}, \perp\}$. An **execution** is a triple $X = ((H, \text{level}), <_{\text{hb}}, <_{\text{ar}})$, where (H, level) is an annotated history, $<_{\text{hb}}$ is a strict partial order over H , and $<_{\text{ar}}$ is a total order over H such that $<_{\text{hb}} \subseteq <_{\text{ar}}$. We refer to $<_{\text{hb}}$ and $<_{\text{ar}}$ as **happens-before** and **arbitration**. ◀

We denote components of an execution as in $X.H$ and use the same notation for similar structures.

A transaction records a set of operations and the order in which the client program invoked them. A history records transactions that committed in a finite database computation. For simplicity we elide the treatment of aborted and ongoing transactions, as well as infinite database computations. Annotated histories enrich histories with a function level that records which transactions the programmer requested to execute under serializability, and which transactions under the weak consistency model offered by the underlying database. Finally, executions enrich annotated histories with a happens-before order and an arbitration order, which declaratively represent internal database processing. Intuitively, $T <_{\text{hb}} S$ means that S is aware of the updates performed by T , and thus the outcome of the operations in S may depend on the effects of T . We call transactions that are not related by happens-before **concurrent**. The relationship $T <_{\text{ar}} S$ means that the versions of objects written by S supersede those written by T . The constraint $<_{\text{hb}} \subseteq <_{\text{ar}}$ ensures that writes by a transaction T supersede those that T is aware of.

We use the set $\{\text{CC}, \text{PC}, \text{PSI}, \text{SI}, \text{SER}\}$ to refer to the consistency models that we treat (§1), and we range over this set by wm . In Figure 4 we specify these consistency models as combination of the axioms in Figure 3, constraining executions. Formally, we let the set of annotated histories allowed by a consistency model wm be given by $\text{hist}(wm) = \{(X.H, X.\text{level}) \mid X \models wm\}$. We now explain the axioms and the anomalies that they (dis)allow. We summarise these anomalies in Figure 2.

Given a total order $< \subseteq A \times A$ and a set $B \subseteq A$, we write $\max(B, <)$ for the element $b \in B$ such that $\forall a \in B. a \leq b$; if $A = \emptyset$, then $\max(B, <)$ is undefined. We define $\min$ in the



■ **Figure 2** Non-serializable executions illustrating anomalies. Boxes represent transactions, and thin arrows between boxes represent the happens-before relation. We omit arbitration edges to avoid clutter. The thick arrows marked wr/rw are explained in §3.

$\forall (E, <_{po}) \in H. \forall e \in E. \forall x, n.$	
$e \vdash \text{read}(x, n) \implies (\text{before}(e, <_{po}, _ x) = \emptyset \vee \max(\text{before}(e, <_{po}, _ x), <_{po}) \vdash _ (_, n))$	(INT)
$\forall T \in H. \forall x, n.$	
$T \vdash \text{read}(x, n) \implies ((\text{before}(T, <_{hb}, \text{write } x) = \emptyset \wedge n = 0) \vee \max(\text{before}(T, <_{hb}, \text{write } x), <_{ar}) \vdash _ (_, n))$	(EXT)
$\forall T, S \in H. (\exists x. T \vdash \text{write}(x, _) \wedge S \vdash \text{write}(x, _)) \implies T = S \vee T <_{hb} S \vee S <_{hb} T$	(CONFLICT)
$<_{ar}; <_{hb} \subseteq <_{hb}$	(PREFIX)
$\forall T, S \in H. T = S \vee T <_{hb} S \vee S <_{hb} T$	(TOTALHB)
$\forall T, S \in H. (\text{level}(T) = \text{level}(S) = \text{SER}) \implies T = S \vee T <_{hb} S \vee S <_{hb} T$	(SERTOTAL)

■ **Figure 3** Consistency axioms constraining an execution $((H, \text{level}), <_{hb}, <_{ar})$.

obvious dual manner. In the following, when we write $\max(B, <)$ or $\min(B, <)$, we assume that they are defined. Given a partial order $< \subseteq A \times A$ and an $a \in A$, we define the downset of a as $\text{before}(a, <) = \{a' \in A \mid a' < a\}$, and let $\text{before}(a, <, \text{op } x) = \text{before}(a, <) \cap \{a' \in A \mid a' \vdash \text{op}(x, _)\}$.

The *internal consistency axiom* INT ensures that, within a transaction, the database provides sequential semantics: in a transaction $(E, <_{po})$, a read event e on an object x returns the value of the last event on x preceding e . The events on x preceding e are given by the set $\text{before}(e, <_{po}, _ x)$. If in $(E, <_{po})$ a read e on x is not preceded by an operation on the same object (i.e., $\text{before}(e, <_{po}, _ x) = \emptyset$), then its value is determined in terms of writes by other transactions, using the *external consistency axiom* EXT. To formulate EXT we lift the $\vdash$ notation to transactions. For given $T = (E, <_{po})$, $x \in \text{Obj}$ and $n \in \mathbb{N}$, we write:

- $T \vdash \text{write}(x, n)$ if $\max(\{e \in E \mid e \vdash \text{write}(x, _)\}, <_{po}) \vdash \text{write}(x, n)$; and
- $T \vdash \text{read}(x, n)$ if $\min(\{e \in E \mid e \vdash _ (x, _)\}, <_{po}) \vdash \text{read}(x, n)$.

According to EXT, if a transaction T reads an object x before writing to it, then the value returned by the read is determined by the transactions that happen before T and that write to x ; the set of such transactions is given by $\text{before}(T, <_{\text{hb}}, \text{write } x)$. If this set is empty, then T reads the initial value 0; otherwise it reads the value written by the transaction from the set that is the last one in $<_{\text{ar}}$. EXT guarantees the *atomic visibility* of a transaction: either all or none of its writes can be visible to another transaction. A detailed discussion on the matter can be found in [12, Section 3].

$\text{CC} \equiv \text{INT} \wedge \text{EXT} \wedge \text{SERTOTAL}$
$\text{PSI} \equiv \text{CC} \wedge \text{CONFLICT}$
$\text{PC} \equiv \text{CC} \wedge \text{PREFIX}$
$\text{SI} \equiv \text{PSI} \wedge \text{PREFIX}$
$\text{SER} \equiv \text{INT} \wedge \text{EXT} \wedge \text{TOTALHB}$

■ **Figure 4** Consistency model definitions.

The axiom SERTOTAL formalises the additional guarantees provided to transactions that the application programmer required to execute on serializability, as recorded by `level`. We discuss this axiom in more detail below; for now we assume $\text{level} = (\lambda T. \perp)$ for all executions.

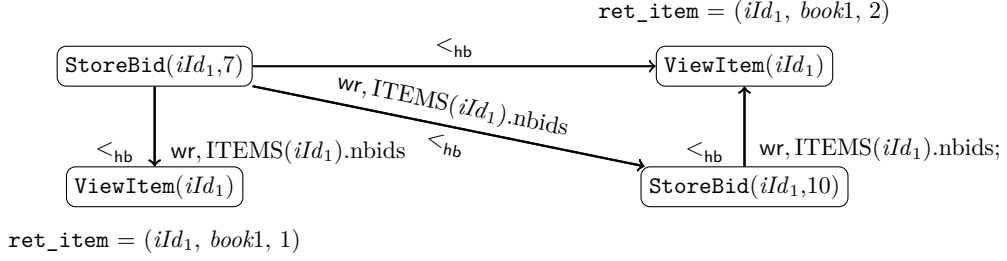
The axioms INT, EXT and SERTOTAL define causal consistency (CC) [22]. This forbids the *causality violation* anomaly in Figure 2(a), where a user sees the review, but not the book it was associated with. This anomaly is forbidden because $<_{\text{hb}}$ is transitive, so we must have $T_1 <_{\text{hb}} T_3$. Since $T_1 <_{\text{ar}} T_2$, the writes by T_2 supersede those by T_1 , and thus EXT implies $T_3 \vdash \text{read}(y, \text{review})$ and $T_3 \vdash \text{read}(x, \text{book})$.

Causal consistency allows the *lost update* anomaly, illustrated by the execution in Figure 2(b). This execution may arise from the programs `ViewItem` and `StoreBid` in Figure 1, which respectively let a user query the information about an item and bid on an item. They access a table ITEMS, whose rows represent items and contain a primary key (`iId`), an item description (`desc`) and the number of existing bids (`nbids`). The anomaly in Figure 2(b) is caused by two invocations of the program `StoreBid` that generate the transactions T_1 and T_2 , meant to increase the number of bids of an item. The two transactions read the initial number of bids for the item, namely 0, and concurrently modify it, resulting in one addition getting lost. This is observed by a third transaction T_3 generated by `ViewItem`. The lost update anomaly is disallowed by the axiom CONFLICT, which guarantees that transactions updating the same object are not concurrent. This axiom rules out any execution with the history in Figure 2(b). We specify parallel snapshot isolation (PSI) [26] by strengthening causal consistency with the axiom CONFLICT. This consistency models allows the *long fork* anomaly given in Figure 2(c), which we discussed in §1.

We specify prefix consistency (PC) [11] and snapshot isolation (SI) [6] by strengthening respectively CC and PSI via the axiom PREFIX: if T observes S , then it also observes all $<_{\text{ar}}$ -predecessors of S , which is formalised using sequential composition $;$ of relations. The axiom PREFIX disallows any execution with the history in Figure 2(c): T_1 and T_2 have to be related by $<_{\text{ar}}$ one way or another; but then by PREFIX, either T_4 has to observe *Alice* or T_3 has to observe *Bob*. Like causal consistency, prefix consistency allows the lost update anomaly in Figure 2(b). Snapshot isolation disallows it, but allows the anomaly of *write skew*, illustrated by the execution in Figure 2(d). This execution could be produced by `RegUser` in Figure 1. The objects x and y correspond to different rows in the table USERS. Two invocations of `RegUser` generate transactions that miss each other's writes and, as a consequence, concurrently register two users with the same nickname.

We define serializability (SER) using the axiom TOTALHB, which requires happens-before to be total. It disallows any execution with one of the histories in Figure 2.

Finally, the consistency models we consider include the axiom SERTOTAL, which requires happens-before to be total on transactions that the programmer marked as serializable.



■ **Figure 5** An execution produced by the programs in Figure 1 and its dynamic dependency graph (the latter explained in §3). We assume $\text{level} = (\lambda T. \perp)$. We omit events inside transactions and only show the parameters and the return values of the corresponding programs. Since $<_{hb} \subseteq <_{ar}$, all the relevant arbitration edges coincide with the happens-before ones, and we omit the $<_{ar}$ label.

For example, in a database providing CC, the history in Figure 2(c) can be disallowed by letting $\text{level}(T_1) = \text{level}(T_2) = \text{SER}$ and $\text{level}(T_3) = \text{level}(T_4) = \perp$. This is because then SERTOTAL forces T_1 and T_2 to be related by happens-before, and therefore either T_3 or T_4 has to observe *both* T_1 and T_2 . We can disallow the history in Figure 2(b) by letting $\text{level}(T_1) = \text{level}(T_2) = \text{SER}$ and $\text{level}(T_3) = \perp$.

As the last example, we consider the execution X in Figure 5, which is produced on a PSI database by the programs `StoreBid` and `ViewItem` in Figure 1: $X \models \text{PSI}$. In X the two transactions due to `StoreBid` submit bids for an item iId_1 : one bid of 7 dollars and one bid of 10 dollars. The other two transactions due to `ViewItem` query the state of the item. The query on the left sees the bid of 7, but not that of 10. **The query on the right sees both bids.** It is easy to check that the history of this execution is serializable. As a matter of fact, the results we develop in the following sections let us show that **any execution produced by the programs `StoreBid` and `ViewItem` under PSI has a serializable history.** Hence, a database can process the corresponding transactions using the PSI concurrency control without exposing any anomalies to its users.

3 Dynamic Robustness Criteria

Our first goal is to define criteria to check whether a single execution X in one of the consistency models that we consider has a serializable history: $X.H \in \text{hist}(\text{SER})$. From these *dynamic* robustness criteria, in the next section we derive *static* criteria to check whether this is the case for all executions of a given application.

Our dynamic criteria are formulated in terms of *dependency graphs*, widely used in the database literature [2]. Let the set of *labels* L be defined as follows: $D = \{(wr, x), (ww, x) \mid x \in \text{Obj}\}$, $L = D \cup \{(rw, x) \mid x \in \text{Obj}\}$. We use λ to range over L and s, t to range over L^* . A graph G is a pair $(H, \longrightarrow)$, where $H \in \text{Hist}$ and $\longrightarrow \subseteq H \times L \times H$. We write $T \xrightarrow{\lambda} S \in G$ in place of $(T, \lambda, S) \in \longrightarrow$. We also use some graph-theoretic notions. A **path** π in G is a non-empty finite sequence of edges $T_0 \xrightarrow{\lambda_0} T_1 \xrightarrow{\lambda_1} \dots \xrightarrow{\lambda_{n-1}} T_n$. In this case we write $\pi \in G$. The path is a **cycle** if $T_0 = T_n$, and it is a **simple** cycle if all other pairs of transactions on it are distinct. We also write $T \xrightarrow{\lambda} S \in \pi$ to mean that the edge $T \xrightarrow{\lambda} S$ appears in the path π . Given a graph $G = (H, \longrightarrow)$, we denote by $\xrightarrow{s}^+$ the least relation such that (i) $T \xrightarrow{\lambda}^+ S$ if $T \xrightarrow{\lambda} S \in G$; and (ii) $T \xrightarrow{\lambda s}^+ S$ whenever $T \xrightarrow{\lambda} T' \in G$ and $T' \xrightarrow{s}^+ S$ for some $T' \in H$. We denote with $\xrightarrow{s}^*$ the reflexive closure of $\xrightarrow{s}^+$, so that $T \xrightarrow{\varepsilon}^* T$ for every $T \in H$. We now define a map from executions into dependency graphs.

► **Definition 2.** The *dynamic dependency graph* of an execution $X = ((H, _), <_{\text{hb}}, <_{\text{ar}})$ is $\text{DDG}(X) = (H, \longrightarrow)$, where for every $x \in \text{Obj}$ the relation $\longrightarrow$ contains the following triples:

read-dependency: $T \xrightarrow{\text{wr}, x} S$ if $S \vdash \text{read}(x, _)$ and $T = \max(\text{before}(S, <_{\text{hb}}, \text{write } x), <_{\text{ar}})$;
write-dependency: $T \xrightarrow{\text{ww}, x} S$ if $T \vdash \text{write}(x, _)$, $S \vdash \text{write}(x, _)$ and $T <_{\text{ar}} S$;
anti-dependency: $T \xrightarrow{\text{rw}, x} S$ if $T \neq S$ and either (i) $T \vdash \text{read}(x, _)$, $S \vdash \text{write}(x, _)$ and $\text{before}(T, <_{\text{hb}}, \text{write } x) = \emptyset$, or (ii) $T' \xrightarrow{\text{wr}, x} T$ and $T' \xrightarrow{\text{ww}, x} S$ for some $T' \in H$. ◀

Thus, $T \xrightarrow{\text{wr}, x} S$ means that S reads T 's write to x (cf. EXT in Figure 3), and $T \xrightarrow{\text{ww}, x} S$ means that S overwrites T 's write to x . The relation $T \xrightarrow{\text{rw}, x} S$ means that S overwrites the write to x read by T (the initial value of an object is overwritten by any write to this object). In Figures 2(c), 2(d) and 5 we draw the dependency graphs with thick edges.

Dependency graphs provide a way to show that executions have serializable histories [2].

► **Lemma 3.** For every X , if $X \models \text{INT} \wedge \text{EXT}$ and $\text{DDG}(X)$ is acyclic, then $X.H \in \text{hist}(\text{SER})$.

For instance, the history in Figure 5 is serializable. The graphs of the executions in Figure 2(c, d) contain cycles and, in fact, the histories of these executions are not serializable.

As we now show, to ensure that the history of an execution X arising from a particular consistency model is serializable, it is enough to check that $\text{DDG}(X)$ does not contain cycles of a particular form, which we call *critical*. This more precise characterisation is instrumental in obtaining our static robustness criteria (§4).

A path π in a dynamic dependency graph G is **chord-free** if, whenever $u \xrightarrow{s}^+ v \in \pi$ for some $s \in L^n$ with $n \geq 2$, we have $\neg(u \longrightarrow v \in G)$. A path π is **rw-minimal** if, whenever $u \xrightarrow{\text{rw}, _} v \in \pi$ and $u \xrightarrow{\lambda} v \in G$, we have $\lambda = (\text{rw}, _)$. The last notion forces us to exclude an rw edge from π if there is another option.

► **Definition 4.** Given an execution X , an edge $T \xrightarrow{\lambda} S \in \text{DDG}(X)$ is **unprotected** if either $X.\text{level}(T) \neq \text{SER}$ or $X.\text{level}(S) \neq \text{SER}$. A cycle $\pi \in \text{DDG}(X)$ among transactions $T_0, T_1, \dots, T_n$ (where $T_0 = T_n$) that is simple, chord-free and rw-minimal is:

CC-critical, if π contains an unprotected edge $T_i \xrightarrow{\text{rw}, _} T_{i+1}$ and an unprotected edge

$T_j \xrightarrow{\lambda} T_{j+1}$ with $i \neq j$ and $\lambda \in \{(\text{ww}, _), (\text{rw}, _)\}$;

PC-critical, if π contains an unprotected edge $T_i \xrightarrow{\text{rw}, _} T_{i+1}$ and at least two adjacent unprotected edges with labels in $\{(\text{ww}, _), (\text{rw}, _)\}$;

PSI-critical, if:

1. π contains at least two unprotected rw edges; and
2. for every $T_i \xrightarrow{\text{rw}, x} T_{i+1}$, $T_j \xrightarrow{\text{rw}, y} T_{j+1} \in \pi$, if $i \neq j$, then $x \neq y$;

SI-critical, if:

1. π contains at least two adjacent unprotected rw edges; and
2. for every $T_i \xrightarrow{\text{rw}, x} T_{i+1}$, $T_j \xrightarrow{\text{rw}, y} T_{j+1} \in \pi$, if $i \neq j$, then $x \neq y$. ◀

The graphs of the executions in Figure 2 (with $\text{level} = (\lambda T. \perp)$) contain critical cycles: (c) contains a PSI-critical cycle, and (d) contains an SI-critical cycle.

► **Theorem 5.** For every wm and X , if $X \models wm$, then $\text{DDG}(X)$ contains a cycle if and only if it contains a wm -critical cycle.

From Theorem 5 and Lemma 3 we obtain our dynamic robustness criterion.

► **Corollary 6.** For every wm and every X , if $X \models wm$ and $\text{DDG}(X)$ contains no wm -critical cycle then $X.H \in \text{hist}(\text{SER})$.

We note that the above robustness criterion for SI is a variant of an existing one [15, 16]. In our setting, it is just a consequence of our novel criterion for PSI.

To prove Theorem 5 we show how the axioms in Figure 3 impact the properties of edges and paths in dependency graphs. First, observe that there is a relation between wr, ww edges and the orders $<_{hb}$ and $<_{ar}$: for every X and every $T, S \in X.H$, the definitions ensure that

$$\begin{aligned} T \xrightarrow{wr, -} S \in \text{DDG}(X) &\implies T <_{hb} S; \\ (T \xrightarrow{wr, -} S \in \text{DDG}(X) \vee T \xrightarrow{ww, -} S \in \text{DDG}(X)) &\implies T <_{ar} S; \\ (X \models \text{CONFLICT} \wedge T \xrightarrow{ww, -} S \in \text{DDG}(X)) &\implies T <_{hb} S. \end{aligned}$$

These implications let us show that, under certain conditions, if two transactions in a dependency graph are connected by a path, then they are also related by happens-before or arbitration.

► **Lemma 7.** *For any X , $s \in L^+$ and $T \xrightarrow{s}^+ S \in \text{DDG}(X)$, if $X \models \text{SERTOTAL}$ then:*

1. *if all the rw and ww edges in $T \xrightarrow{s}^+ S$ are protected then $T <_{hb} S$;*
2. *if all the rw edges in $T \xrightarrow{s}^+ S$ are protected then $T <_{ar} S$;*
3. *if $X \models \text{CONFLICT}$ and all the rw edges in $T \xrightarrow{s}^+ S$ are protected then $T <_{hb} S$.*

The following lemma shows that, if $T \xrightarrow{rw, x} S$, then T cannot happen-before S : in this case T would have to read a value at least as up-to-date as that written by S , contradicting the definition of anti-dependencies.

► **Lemma 8.** $\forall X. \forall x \in \text{Obj}. \forall T, S \in X.H. T \xrightarrow{rw, x} S \in \text{DDG}(X) \implies S \not<_{hb} T \wedge T \vdash \text{read}(x, _) \wedge S \vdash \text{write}(x, _).$

Proof of Theorem 5. The *if* implication is obvious, so let us prove the *only if* implication. Suppose that the graph $\text{DDG}(X)$ contains a cycle π' . From π' we can easily build a cycle

$$\pi = T_0 \xrightarrow{\lambda_0} T_1 \xrightarrow{\lambda_1} \dots \xrightarrow{\lambda_{n-1}} T_n \quad (\text{where } T_0 = T_n, n \geq 2) \quad (1)$$

in $\text{DDG}(X)$ that is simple, chord-free and rw -minimal. The argument now is a case analysis on the wm . Here we consider only CC and PSI and defer the full proof to [7].

Case of $wm = \text{CC}$. Lemma 7(2) implies that π contains at least one unprotected rw edge, for otherwise $T_0 <_{ar} T_0$, contradicting the irreflexivity of $X.<_{ar}$. Let this edge be $T_i \xrightarrow{rw, -} T_{i+1}$. Then Lemma 8 ensures that $T_{i+1} \not<_{hb} T_i$. Since π contains the non-empty path $T_{i+1} \xrightarrow{-}^+ T_i$, Lemma 7(1) implies that on this path there is at least one unprotected edge $T_j \xrightarrow{\lambda} T_{j+1}$ with $\lambda \in \{(ww, _), (rw, _)\}$ and $i \neq j$. It follows that π is CC-critical.

Case of $wm = \text{PSI}$. First we prove that the cycle π contains at least two unprotected edges rw edges. Since $X \models \text{PSI}$, we know that $X \models \text{CC}$. Thus, the previous argument ensures that the cycle π contains at least one unprotected rw edge, say $T_i \xrightarrow{rw, x} T_{i+1}$. Suppose that π contains exactly one such edge. Since $X \models \text{CONFLICT}$, Lemma 7(3) now ensures $T_{i+1} <_{hb} T_i$. But by Lemma 8, $T_i \xrightarrow{rw, x} T_{i+1}$ implies $T_{i+1} \not<_{hb} T_i$. The resulting contradiction shows that π must contain at least two unprotected rw edges.

Now we have to prove

$$\forall T_i \xrightarrow{rw, x} T_{i+1}, T_j \xrightarrow{rw, y} T_{j+1} \in \pi. i \neq j \implies x \neq y. \quad (2)$$

Suppose that π does not satisfy (2). Then, as $<_{ar}$ is total, Definition 2 guarantees that we have either $T_{i+1} \xrightarrow{ww, x} T_{j+1}$ or $T_{j+1} \xrightarrow{ww, x} T_{i+1}$. Since π is a simple cycle, in the first

case we contradict either that π is chord-free or that π is rw-minimal. In the second case, we have either (a) $T_{j+1} = T_i$ or (b) $T_{j+1} \neq T_i$. If (a) holds, then we contradict that π is rw-minimal, because $T_i \xrightarrow{\text{rw}, -} T_{i+1} \in \pi$ and $T_i \xrightarrow{\text{ww}, -} T_{i+1}$. If (b) holds, then the sub-path $T_{j+1} \xrightarrow{s, +} T_{i+1}$ of π contains at least two edges and it is chord-free by construction. But this contradicts $T_{j+1} \xrightarrow{\text{ww}, x} T_{i+1}$. It follows that π satisfies (2) above, and thus it is a PSI-critical cycle. $\blacktriangleleft$

4 Static Robustness Criteria

We now illustrate how the dynamic robustness criteria (Corollary 6) can be lifted to static criteria, which allow programmers to analyse the behaviour of their applications and which can serve as a basis for static analysis tools.

We define an *application* $\mathcal{A}$ by a set of *transactional programs* $\mathbf{f}_i$, giving the code of its transactions: $\mathcal{A} = \{\mathbf{f}_1, \dots, \mathbf{f}_n\}$ (e.g., see Figure 1). As is standard in the database literature [16], this abstracts from the rest of the application logic to focus on the parts that directly interact with the database. We call a pair $I = (\mathbf{f}, \bar{\mathbf{v}})$ of a program and a vector of its actual parameters a *program instance*. An *application instance* $\mathcal{I}$ is a set of program instances, and an *annotated application instance* is a pair $(\mathcal{I}, \text{level}_{\mathcal{S}})$, where $\text{level}_{\mathcal{S}} : \mathcal{I} \rightarrow \{\text{SER}, \perp\}$ defines which programs the programmer requested to execute under serializability. We first formulate criteria for checking the robustness of a particular annotated application instance, resulting from running a set of transactional programs with given parameters. We then sketch how these criteria can be generalised to whole applications.

We aim to illustrate the ideas for lifting dynamic robustness criteria to static ones in the simplest form. To this end, we abstract from the syntax of the programming language and assume that we are only given approximate information about the set of objects read or written by each transactional program. Namely, we assume a function rwsets that maps every program instance I to a triple $\text{rwsets}(I) = (R^{\diamond}, W^{\diamond}, W^{\square})$. Informally, $R^{\diamond}$ and $W^{\diamond}$ are the sets of all the objects that *may* be read or written in some execution of I , and $W^{\square}$ is a set of the objects that *must* be written in any execution of I , with the proviso that $W^{\square} \subseteq W^{\diamond}$. For instance, for $I = (\text{StoreBid}, \langle iId_1, 7 \rangle)$ (Figure 1) we have

$$\text{rwsets}(I) = (\{\text{ITEMS}(iId_1).\text{nbids}\}, \{\text{ITEMS}(iId_1).\text{nbids}, \text{BIDS}(*).*\}, \{\text{ITEMS}(iId_1).\text{nbids}\})$$

where $*$ means “all fields” or “all rows”.

To formalise the meaning of the read/write sets, we define a relation that determines if a history can be produced by a given $\mathcal{I}$. We let $T \Vdash I$ for $\text{rwsets}(I) = (R^{\diamond}, W^{\diamond}, W^{\square})$, if:

- (i) $T \vdash \text{write}(x, _) \implies x \in W^{\diamond}$;
- (ii) $T \vdash \text{read}(x, _) \implies x \in R^{\diamond}$;
- (iii) $x \in W^{\square} \implies T \vdash \text{write}(x, _)$.

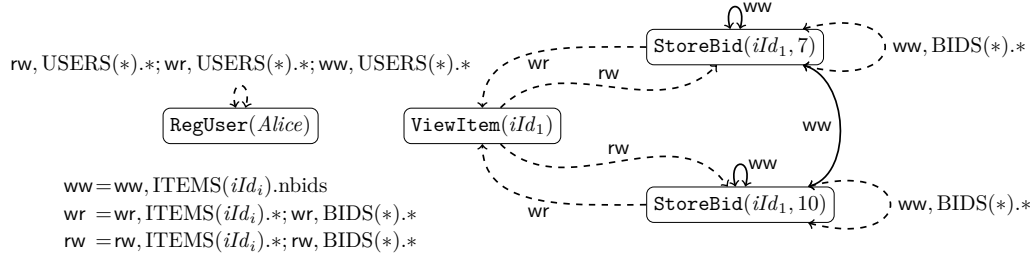
We lift the relation $\Vdash$ to annotated histories and annotated application instances:

$$(H, \text{level}) \Vdash (\mathcal{I}, \text{level}_{\mathcal{S}}) \iff \forall T \in H. \exists I \in \mathcal{I}. T \Vdash I \wedge \text{level}_{\mathcal{S}}(I) = \text{level}(T).$$

Note that the definition of $\Vdash$ allows multiple transactions in H to be associated to a single I in $\mathcal{I}$. For example, we have $(H, (\lambda T. \perp)) \Vdash (\mathcal{I}, (\lambda I. \perp))$ for the history H in Figure 5 and

$$\mathcal{I} = \{(\text{RegUser}, \text{Alice}), (\text{ViewItem}, iId_1), (\text{StoreBid}, \langle iId_1, 7 \rangle), (\text{StoreBid}, \langle iId_2, 10 \rangle)\}. \quad (3)$$

We formulate our robustness criteria by adapting modal transition systems [21].



■ **Figure 6** The static dependency graph $\text{SDG}(\mathcal{I})$ of the application instance $\mathcal{I}$ defined in (3). We draw may edges with dashed arrows, and must edges with solid arrows.

► **Definition 9.** The *static dependency graph* of an application instance $\mathcal{I}$ is a triple $\text{SDG}(\mathcal{I}) = (\mathcal{I}, \dashrightarrow, \hookrightarrow)$, where the relations $\dashrightarrow$ and $\hookrightarrow$ are defined as follows. For every $I, J \in \mathcal{I}$, if $\text{rwsets}(I) = (W_I^\diamond, R_I^\diamond, W_I^\square)$ and $\text{rwsets}(J) = (W_J^\diamond, R_J^\diamond, W_J^\square)$, then:

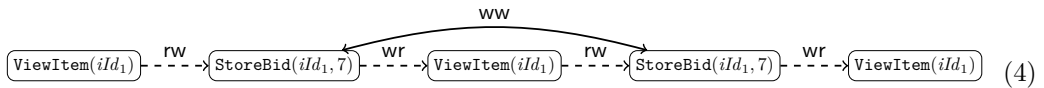
$$\begin{aligned} I \dashrightarrow^{wr, x} J &\iff x \in W_I^\diamond \cap R_J^\diamond; & I \dashrightarrow^{ww, x} J &\iff x \in W_I^\diamond \cap W_J^\diamond; \\ I \dashrightarrow^{rw, x} J &\iff x \in R_I^\diamond \cap W_J^\diamond; & I \hookrightarrow^{ww, x} J &\iff x \in W_I^\square \cap W_J^\square. \end{aligned}$$

Figure 6 shows the static dependency graph of the $\mathcal{I}$ in (3). Informally, the edges of the static dependency graph $\text{SDG}(\mathcal{I})$ describe possible dependencies between transactions in executions produced by $\mathcal{I}$: an edge $I \dashrightarrow J$ represents a dependency that may exist, and an edge $I \hookrightarrow J$ a dependency that must exist. Formally, given an annotated application instance $(\mathcal{I}, \text{level}_S)$, we say that the pair $(\text{DDG}(X), X.\text{level})$ is *over-approximated* by the pair $(\text{SDG}(\mathcal{I}), \text{level}_S)$, written $(\text{DDG}(X), X.\text{level}) \triangleleft (\text{SDG}(\mathcal{I}), \text{level}_S)$, if for some total function $f : X.H \rightarrow \mathcal{I}$ we have:

1. $\forall T \xrightarrow{\lambda} S \in \text{DDG}(X). f(T) \dashrightarrow f(S) \in \text{SDG}(\mathcal{I})$;
2. $\forall I \hookrightarrow J \in \text{SDG}(\mathcal{I}).$
 $\forall T \in f^{-1}(I). \forall S \in f^{-1}(J). T \xrightarrow{\lambda} S \in \text{DDG}(X) \vee S \xrightarrow{\lambda} T \in \text{DDG}(X)$; and
3. $\text{level}(T) = \text{level}_S(f(T))$.

► **Lemma 10.** $\forall X. \forall (\mathcal{I}, \text{level}_S). (X.H, X.\text{level}) \Vdash (\mathcal{I}, \text{level}_S) \implies$
 $(\text{DDG}(X), \text{level}) \triangleleft (\text{SDG}(\mathcal{I}), \text{level}_S)$.

We now formulate our static robustness criteria by using the same notions of paths and cycles for static dependency graphs as for dynamic ones (§3). Given a pair $(\mathcal{I}, \text{level}_S)$ and a cycle in $\pi \in \text{SDG}(\mathcal{I})$ among program instances $I_0, I_1, \dots, I_n$ (where $I_0 = I_n$), we say that an rw edge $I_i \dashrightarrow^{rw, x} I_{i+1} \in \pi$ is *critical in* π , if for all I_l, I_m in π such that $l \neq m$ and for all $t, t' \in D^*$ such that $I_l \dashrightarrow^t I_i \in \pi$ and that $I_{i+1} \dashrightarrow^{t'} I_m \in \pi$, we have $\neg(I_l \xrightarrow{ww} I_m)$. For example, the graph in Figure 6 contains the following cycle π , in which the left-most rw edge is critical, while the right-most rw edge is not critical:



► **Definition 11.** Given a pair $(\mathcal{I}, \text{level}_S)$, an edge $I_i \dashrightarrow I_{i+1} \in \text{SDG}(\mathcal{I})$ is *unprotected* if either $\text{level}_S(I_i) \neq \text{SER}$ or $\text{level}_S(I_{i+1}) \neq \text{SER}$. A cycle $\pi \in \text{SDG}(\mathcal{I})$ among program instances $I_0, I_1, \dots, I_n$ (where $I_0 = I_n$) is:

CC-critical, if π contains an unprotected edge $I_i \xrightarrow{\text{rw}, _} I_{i+1}$ and an unprotected edge $I_j \xrightarrow{\lambda} I_{j+1}$ with $i \neq j$ and $\lambda \in \{(\text{ww}, _), (\text{rw}, _)\}$;

PC-critical, if π contains an unprotected edge $I_i \xrightarrow{\text{rw}, _} I_{i+1}$ and at least two adjacent unprotected edges with labels in $\{(\text{ww}, _), (\text{rw}, _)\}$. ;

PSI-critical, if:

1. π contains at least two unprotected critical rw edges; and
2. for every $I_i \xrightarrow{\text{rw}, x} I_{i+1}$, $I_j \xrightarrow{\text{rw}, y} I_{j+1} \in \pi$, if $i \neq j$, then $x \neq y$;

SI-critical, if:

1. π contains at least two adjacent unprotected critical rw edges; and
2. for every $I_i \xrightarrow{\text{rw}, x} I_{i+1}$, $I_j \xrightarrow{\text{rw}, y} I_{j+1} \in \pi$, if $i \neq j$, then $x \neq y$. ◀

Note that, unlike a critical cycle in a dynamic dependency graph (Definition 4), a critical cycle in a static graph does not have to be simple. The following lemma states that $\triangleleft$ preserves critical cycles.

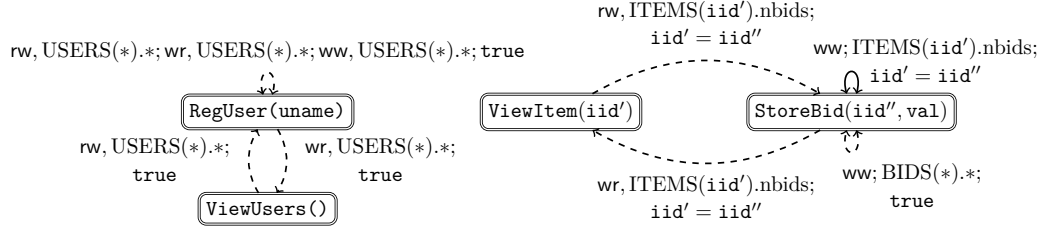
► **Lemma 12.** *For every wm , (G, level) and (F, level_S) such that $(G, \text{level}) \triangleleft (F, \text{level}_S)$, if G contains a wm -critical cycle, then F contains a wm -critical cycle.*

Lemmas 10 and 12 (which are proven in [7]) and Corollary 6 establish our static criteria.

► **Theorem 13.** *For every (H, level) , $(\mathcal{I}, \text{level}_S)$ and wm , if $\text{SDG}(\mathcal{I})$ contains no wm -critical cycles and $(H, \text{level}) \in \text{hist}(wm)$, then whenever $(H, \text{level}) \Vdash (\mathcal{I}, \text{level}_S)$, we have $H \in \text{hist}(\text{SER})$.*

For example, let $\text{level}_S = (\lambda I. \perp)$ and consider $\mathcal{I}$ defined by (3). The corresponding static dependency graph in Figure 6 contains PSI-critical cycles, one of which is obtained by following twice the loop $\text{RegUser}(\text{Alice}) \xrightarrow{\text{rw}, \text{USERS}(_).name} \text{RegUser}(\text{Alice})$. Indeed, as we explained in §2, the annotated instance $(\mathcal{I}, \text{level}_S)$ is not robust against PSI, because it can produce the write skew anomaly in Figure 2(d). Now let $\text{level}'_S(\text{RegUser}, \text{Alice}) = \text{SER}$ and $\text{level}'_S(_) = \perp$ otherwise. Figure 6 contains the static dependency graph corresponding to the annotated instance $(\mathcal{I}, \text{level}'_S)$. This graph does not contain PSI-critical cycles. To see why, observe that in the graph there are only two kinds of cycles: the ones due to the self-loop on the node $\text{RegUser}(\text{Alice})$, and the ones that connect nodes in $\{\text{StoreBid}(iId_1, 7), \text{StoreBid}(iId_1, 10), \text{ViewItem}(iId_1)\}$. The cycles of the first kind contain only protected rw edges thanks to level'_S , while the cycles of the second kind contain at most one critical rw edge, as sketched in (4) above. It follows that no cycle is PSI-critical, and thus by executing only the RegUser transaction on serializability, we make $\mathcal{I}$ robust. However, the graph contains a CC-critical cycle, namely the one shown in (4) above. It is CC-critical for its two rw edges are unprotected. As we explained in §2, under CC $\mathcal{I}$ may produce the lost update anomaly in Figure 2(b), and it is unsafe to run $\mathcal{I}$ over a CC database.

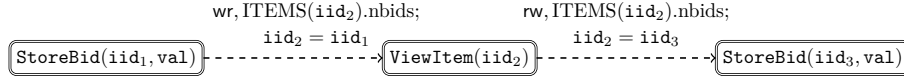
Analysing Whole Applications. The static criteria in Theorem 13 allow a programmer to analyse the robustness of a given application instance. Analysing an application completely using the theorem requires considering an infinite number of its instances, a task best done by an automatic static analysis tool. We now sketch how ideas from abstract interpretation [13, 25] can be used to finitely represent and analyse the set of all instances of an application. In the future, this can pave the way to automating our robustness criteria in static analysis tools. Due to space constraints, we only present the concepts by an example.



■ **Figure 7** The summary dependency graph $\text{SDG}(\mathcal{A})$, where $\mathcal{A}$ contains all the transactional programs in Figure 1.

We associate an application $\mathcal{A}$ with a *summary dependency graph* $\text{SDG}(\mathcal{A})$ that summarises the static graphs of all the instances of $\mathcal{A}$. In Figure 7 we show the summary dependency graph $\text{SDG}(\mathcal{A})$ for the application in Figure 1. Every program in $\mathcal{A}$ yields a *summary node* in the graph $\text{SDG}(\mathcal{A})$, representing all instances of the program. Every edge in $\text{SDG}(\mathcal{A})$ is a *summary edge*, summarising the possible dependencies between the corresponding programs. It is annotated by a constraint relating the actual parameters of the incident programs between which the dependency exists. For example, the summary edge from `ViewItem` to `StoreBid` in Figure 7 means that for every instance $\mathcal{I}$ of $\mathcal{A}$ we have $\text{StoreBid}(iid_1, _) \xrightarrow{\text{rw}, x} \text{ViewItem}(iid_2)$ in $\text{SDG}(\mathcal{I})$ iff $x \in \{\text{BIDS}(*).iid, \text{BIDS}(*).val, \text{ITEMS}(iid_1).\text{nbids}\}$ and $iid_1 = iid_2$. Similarly, the `ww` edge incident to `StoreBid` means that we have $\text{StoreBid}(iid_1, _) \xleftarrow{\text{ww}, x} \text{StoreBid}(iid_2, _)$ in $\text{SDG}(\mathcal{I})$ iff $x = \text{ITEMS}(iid_1).\text{nbids}$ and $iid_1 = iid_2$.

Definition 11 carries over to summary graphs of applications by taking into account the constraints on summary edges when checking whether a given `rw` edge is critical in a given cycle π , and whether the objects that appear on the `rw` edges of π are different. For example, consider the following cycle in the graph in Figure 7:



We consider the `rw` edge on this cycle not critical. This is because the constraints on the edges in the cycle imply $iid_1 = iid_3$, which satisfies the constraint on the must `ww` edge between `StoreBid` programs in Figure 7. For any annotated instance $(\mathcal{I}, \text{level}_S)$ of the application $\mathcal{A}$ in Figure 1, using the adjusted Definition 11 we can check that, if level_S maps the instances of `RegUser` and `ViewUser` in $\mathcal{I}$ to `SER`, then $(\mathcal{I}, \text{level}_S)$ is robust against PSI^1 .

5 Related Work

In the setting of databases, application robustness was first investigated by Fekete et al. [16], who proposed a criterion for robustness against snapshot isolation (SI) [6]. Fekete then extended the criterion to SI databases allowing the programmer to request serializability for certain transactions [15], a mechanism that we also consider. Our criterion is formulated in a way similar to that of Fekete et al., using dependency graphs [2]. However, in contrast to

¹ Marking `ViewUser` as `SER` is actually unnecessary to make this application robust under PSI , because the graph $\text{SDG}(\mathcal{A})$ contains an edge $\text{ViewUser}() \xrightarrow{\text{rw}, \text{USERS}(*).*} \text{RegUser}(\text{uname})$ which does not exist in the dependency graph of any execution of $\mathcal{A}$. This can be addressed by a more precise static analysis.

their work, we consider more subtle models of parallel snapshot isolation, prefix consistency and causal consistency, which allow more anomalies than SI. The method we use is also different from that of Fekete et al. They consider an operational specification of SI [6], which makes the proof of the robustness criterion highly involved. In contrast, we benefit from using declarative specifications that achieve conciseness by exploiting atomic visibility of transactions [12]. This allows us to come up with robustness criteria more systematically.

Robustness has also been investigated for applications running on weak shared-memory models of common multiprocessors and programming languages (e.g., [9]). However, this line of work has not considered applications using transactions. Transactions complicate the consistency model semantics, which makes establishing robustness criteria more challenging.

Serializability of transactions in an application simplifies establishing its correctness properties, but is not necessary for this. Thus, an alternative approach to establishing application correctness is to prove its desired properties directly, without requiring the transactions to produce only serializable behaviours. Corresponding methods have been proposed for ANSI SQL isolation levels and SI by Lu et al. [23], and for PSI and some of other recent models by Gotsman et al. [18]. Such methods are complementary to ours: the conditions they require can be satisfied by more applications, but are more difficult to check than robustness.

6 Conclusion

In this paper we have made the first steps towards understanding the impact of recently-proposed transactional consistency models for large-scale databases on the correctness properties of applications using them. To this end, we have proposed criteria for checking when an application using a weak consistency model exhibits only strongly consistent behaviours. This enables programmers to check that application correctness will be preserved for a particular choice of a consistency model or transactions to be executed under serializability.

The robustness result of Fekete et al. for SI has previously given rise to automatic tools for statically detecting anomalies in applications [19]. Our work could form a basis for similar advances in databases providing weaker consistency models. Our dynamic robustness criteria are also of an independent interest: apart from serving as a basis for static analysis, such criteria can be used to optimise run-time monitoring algorithms [24, 28].

In establishing our robustness criteria, we have followed a systematic approach that exploits axiomatic specifications [12]: using the axioms of a consistency model, we have characterised the cycles allowed in dependency graphs of executions on the model, and exploited the characterisations to provide sound static analysis techniques. We hope that this method will be applicable to other consistency models being proposed for large-scale databases.

References

- 1 D. Abadi. Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *IEEE Computer*, 45(2), 2012.
- 2 A. Adya. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*. Ph.D., MIT, Cambridge, MA, USA, Mar. 1999.
- 3 M. S. Ardekani, P. Sutra, and M. Shapiro. Non-monotonic snapshot isolation: Scalable and strong consistency for geo-replicated transactional systems. In *SRDS*, 2013.
- 4 P. Bailis, A. Davidson, A. Fekete, A. Ghodsi, J. M. Hellerstein, and I. Stoica. Highly Available Transactions: virtues and limitations. In *VLDB*, 2014.

- 5 P. Bailis, A. Fekete, A. Ghodsi, J. M. Hellerstein, and I. Stoica. Scalable atomic visibility with RAMP transactions. In *SIGMOD*, 2014.
- 6 H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O’Neil, and P. O’Neil. A critique of ANSI SQL isolation levels. In *SIGMOD*, 1995.
- 7 G. Bernardi and A. Gotsman. Robustness against consistency models with atomic visibility (extended version). Available from <http://software.imdea.org/~gotsman/>.
- 8 P. A. Bernstein, V. Hadzilacos, and N. Goodman. *Concurrency Control and Recovery in Database Systems*. 1987.
- 9 A. Bouajjani, E. Derevenetc, and R. Meyer. Checking and enforcing robustness against TSO. In *ESOP*, 2013.
- 10 S. Burckhardt, D. Leijen, M. Fähndrich, and M. Sagiv. Eventually consistent transactions. In *ESOP*, 2012.
- 11 S. Burckhardt, S. Leijen, J. Protzenko, and M. Fähndrich. Global sequence protocol: A robust abstraction for replicated shared state. In *ECOOOP*, 2015.
- 12 A. Cerone, G. Bernardi, and A. Gotsman. A framework for transactional consistency models with atomic visibility. In *CONCUR*, 2015.
- 13 P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, 1977.
- 14 G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels. Dynamo: Amazon’s highly available key-value store. In *SOSP*, 2007.
- 15 A. Fekete. Allocating isolation levels to transactions. In *PODS*, 2005.
- 16 A. Fekete, D. Liarokapis, D. J. O’Neil, P. O’Neil, and D. Shasha. Making snapshot isolation serializable. *ACM Trans. Database Syst.*, 30(2), 2005.
- 17 S. Gilbert and N. Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 2002.
- 18 A. Gotsman, H. Yang, C. Ferreira, M. Najafzadeh, and M. Shapiro. ’Cause I’m strong enough: reasoning about consistency choices in distributed systems. In *POPL*, 2016.
- 19 S. Jorwekar, A. Fekete, K. Ramamritham, and S. Sudarshan. Automating the detection of snapshot isolation anomalies. In *VLDB*, 2007.
- 20 A. Lakshman and P. Malik. Cassandra: A decentralized structured storage system. *SIGOPS Oper. Syst. Rev.*, 44(2), 2010.
- 21 K. G. Larsen and B. Thomsen. A modal process logic. In *LICS*, 1988.
- 22 W. Lloyd, M. J. Freedman, M. Kaminsky, and D. G. Andersen. Don’t settle for eventual: scalable causal consistency for wide-area storage with COPS. In *SOSP*, 2011.
- 23 S. Lu, A. J. Bernstein, and P. M. Lewis. Correct execution of transactions at different isolation levels. *IEEE Trans. Knowl. Data Eng.*, 16(9), 2004.
- 24 D. R. K. Ports and K. Grittnr. Serializable snapshot isolation in PostgreSQL. *Proc. VLDB Endow.*, 5(12), Aug. 2012.
- 25 M. Sagiv, T. Reps, and R. Wilhelm. Parametric shape analysis via 3-valued logic. *ACM Trans. Program. Lang. Syst.*, 24(3):217–298, 2002.
- 26 Y. Sovran, R. Power, M. K. Aguilera, and J. Li. Transactional storage for geo-replicated systems. In *SOSP*, 2011.
- 27 D. Terry, V. Prabhakaran, R. Kotla, M. Balakrishnan, and M. K. Aguilera. Transactions with consistency choices on geo-replicated cloud storage. Technical Report MSR-TR-2013-82, Microsoft Research, 2013.
- 28 K. Zellag and B. Kemme. Consistency anomalies in multi-tier architectures: Automatic detection and prevention. *The VLDB Journal*, 23(1), 2014.